

JEGYZŐKÖNYV

Modern adatbázis rendszerek MSc

MongoDB – NoSQL alapok

Készítette: Csákó Balázs

Neptunkód: IOMAG5

Dátum: 2026. május 07.

Contents

1	Elmélet röviden	2
1.1	Mi az a MongoDB?	2
1.2	CRUD műveletek és operátorok	2
1.3	Indexek	2
1.4	Aggregációs pipeline	2
2	1. feladat – Előadás és mongosh gyakorlat	3
2.1	A feladatok tervezésének lépései	3
2.2	A futtatás eredménye	3
3	2. feladat – MongoDB Java Driver	4
3.1	A feladat elmélete	4
3.2	Gyakorlati feladatok	4

1 Elmélet röviden

1.1 Mi az a MongoDB?

A MongoDB egy nyílt forráskódú, dokumentum-orientált NoSQL adatbázis-kezelő, amely BSON (Binary JSON) formátumban tárolja az adatokat. A relációs rendszerekkel ellentétben itt nincsenek előre definiált sémák: ugyanazon gyűjteményen belül minden dokumentum eltérő mezőkkel rendelkezhet. Ez a rugalmasság különösen előnyös a gyorsan változó követelményeknél és a prototípusfejlesztésnél.

A BSON a JSON-nál bővebb típuskészletet kínál (Date, ObjectId, Decimal128, Int32/Int64), ami lehetővé teszi a hatékony belső tárolást és indexelést. Minden dokumentum automatikusan kap egy egyedi `_id` mezőt (ObjectId), és a rendszer horizontálisan skálázható sharding segítségével.

A relációs világból ismert fogalmak közvetlenül megfeleltethetőek:

Relációs (Oracle/SQL)	MongoDB
Adatbázis	Database
Tábla	Collection (gyűjtemény)
Sor / Oszlop	Document / Field
PRIMARY KEY	<code>_id</code>
JOIN	<code>\$lookup</code> vagy beágyazás

1.2 CRUD műveletek és operátorok

A MongoDB négy alapl műveletet támogat: létrehozás (`insertOne`, `insertMany`), olvasás (`find`, `findOne`), módosítás (`updateOne`, `updateMany`) és törlés (`deleteOne`, `deleteMany`). Minden művelet első paramétere egy JSON szűrő, amelyben összehasonlító (`$gt`, `$gte`, `$lt`, `$lte`, `$eq`), logikai (`$or`, `$and`, `$not`) és módosító (`$set`, `$push`, `$inc`) operátorok használhatók.

1.3 Indexek

Index nélkül a motor COLLSCAN-t végez, azaz minden dokumentumot végigolvas. Egy `createIndex` hívás után IXSCAN-ra vált, ami nagyságrendekkel gyorsabb. Az `explain("executionStats")` paranccsal közvetlenül összehasonlítható a két módszer teljesítménye. Összetett indexek is létrehozhatók, például egy növekvő és egy csökkenő mező kombinációjával.

1.4 Aggregációs pipeline

Az aggregáció a MongoDB legerősebb funkciója: lépésenkénti (stage-enkénti) adatfeldolgozást valósít meg, ahol minden stage átalakítja és továbbadja az adatokat. A legfontosabb stage-ek: `$match` (szűrés, mint a WHERE), `$group` (csoportosítás, mint a GROUP BY), `$sort` (rendezés), `$project` (mezőkiválasztás, mint a SELECT), `$unwind` (tömb kibontása külön dokumentumokká) és `$lookup` (gyűjtemények összekapcsolása, mint a LEFT JOIN). Az akkumulátor-operátorok (`$sum`, `$avg`, `$min`, `$max`, `$first`) a csoportosításon belüli számításokat végzik.

2 1. feladat – Előadás és mongosh gyakorlat

A félév során választott témaköröm a MongoDB volt. A feladat része volt egy elméleti előadás elkészítése, a hozzá tartozó gyakorlati feladatsor összeállítása (mintáadatokkal és megoldásokkal), valamint a saját XML adatbázisra adaptált mongosh lekérdezések elkészítése.

2.1 A feladatok tervezésének lépései

Az előadás 10 diából állt és a következő logikai ívet követte: először az elméleti alapokat mutattam be (dokumentum-orientált modell, BSON, fogalmi megfeleltetés), majd a telepítési lépéseket (MongoDB Server, mongosh, mongoimport). Ezután a CRUD műveleteket demonstráltam élő példákon, kitértem az indexek hatására és az `explain` használatára, végül az aggregációs pipeline-t mutattam be több példán keresztül.

A feladatsorhoz két JSON adatfájlt készítettem: `ettermek.json` (6 étterem, beágyazott cím objektummal, csillagszámmal és specialitások tömbbel) és `szakacsok.json` (4 szakács, végzettségek tömbbel és étterem-hivatkozással). A feladatok 5 blokkban fedik le a tananyagot: környezet-előkészítés és import, CRUD műveletek és DML, indexek vizsgálata, aggregációs pipeline-ok, és egy önálló feladat.

A **saját adatbázis** a korábban elkészített könyvtári XML-ből származik. Két gyűjteményt hoztam létre: `konyvek` és `tagok`. A megfeleltetés az órai példához képest:

Órai (vendeglátas)	Saját (konyvtar)
ettermek (nev, csillag, varos)	konyvek (cim, kiadas_eve, kiado)
szakacsok (nev, eletkor, etterem_nev)	tagok (nev, kor, kedvenc_kiado)
specialitasok	mufajok
vegzettsegek	erdeklodesek

A lekérdezések lefedik az összes órai feladattípust: `mongoimport`-tal feltöltöttem a JSON fájlokat, `insertOne`-nal kézzel is hozzáadtam egy könyvet, majd projekciós, szűrési (`$gte`, `$or`), rendezési és limitálási lekérdezéseket írtam. A DML részben `updateOne`-nal módosítottam kiadási évet, `$push`-sal bővítettem műfaj tömböt, és `deleteMany`-vel töröltem a 2005 előtti könyveket. Az index-feladatban `createIndex` előtt és után is futtattam `explain`-t, hogy látható legyen a `COLLSCAN`→`IXSCAN` váltás. Végül négy aggregációs pipeline-t készítettem: csoportosítás kiadónként átlaggal, `$unwind` az érdeklődésekre, `$lookup` a tagok és könyvek között, és a legidősebb tag meghatározása kiadónként a `$sort+$group+$first` mintával.

A teljes mongosh forrás a mellékelt `sajat.txt` fájlban található.

2.2 A futtatás eredménye

A lekérdezések a várt eredményeket adták. A szűrések helyesen szűkítették az eredményhalmazt, az `explain` az index létrehozása után `IXSCAN`-t mutatott. Az aggregációk közül például a kiadónkénti csoportosítás két csoportot adott vissza (Prentice Hall: 1 könyv, átlag 2012; Teszt Kiadó: 2 könyv, átlag 2011.5), az érdeklődések kibontása után a `sci-fi` volt a leggyakoribb (2 tag), a legidősebb tag kiadónként pedig Kiss Péter (41 év, Prentice Hall) és Teszt Tímea (35 év, Teszt Kiadó) lett.

A `$lookup` pipeline sikeresen összekapcsolta a tagokat a kedvenc kiadójuk könyveivel, és a 2011 utáni szűrés után csak a releváns találatokat adta vissza.

3 2. feladat – MongoDB Java Driver

A második feladat a MongoDB kezelése Java kódból, a hivatalos `mongodb-driver-sync` könyvtár segítségével, a 11. gyakorlat mintájára.

3.1 A feladat elmélete

A MongoDB Java Driver a hivatalos szinkron kliens, amely a `MongoClient` osztályon keresztül csatlakozik a szerverhez. A dokumentumokat a `Document` osztály reprezentálja, amely lényegében egy rendezett kulcs-érték térkép (BSON). A lekérdezésekhez és aggregációkhoz statikus segédmetódusok állnak rendelkezésre: a `Filters` osztály (pl. `eq`, `gt`, `lt`), az `Aggregates` (pl. `match`, `group`, `lookup`) és az `Accumulators` (pl. `avg`, `sum`).

A fejlesztési környezet IntelliJ IDEA volt, Maven projekttel. Az adatfájlok (`konyvek.json`, `tagok.json`) `TempList` wrapperben érkeznek, így a programnak kezelnie kellett az azonosítók átnevezését (`kkod`→`_id`, `tkod`→`_id`) és a string-ek számmá alakítását (`kiadas_eve`, `kor`, `tagdij`).

3.2 Gyakorlati feladatok

A program három fő blokkra tagolódik, a 11. gyakorlat struktúráját követve:

Collection létrehozása és feltöltése. Először 3 könyvet szűrtam be kézzel a `Document` konstruktorral (`_id`: `k4`, `k5`, `k6`), majd a JSON fájlokat olvastam be a `Files.readString` segítségével. A beolvasott dokumentumokat Java Stream-mel alakítottam át: az eredeti kódmezőt (`kkod`) `_id`-vé neveztem át, a szöveges számmezőket (`kiadas_eve`) `Integer.parseInt`-tel konvertáltam, és az `insertMany`-vel szűrtam be őket.

Lekérdezések és aggregáció. Öt lekérdezést implementáltam: az összes könyv listázása (`find`), egy könyv keresése `_id` alapján (`eq` szűrővel), a 35 évnél idősebb tagok szűrése (`gt`), a 41 éves tag és kedvenc kiadójának könyvei közötti összekapcsolás (`match` + `lookup` pipeline), valamint a tagok átlagos életkora (`group` + `avg` akkumulátor). A `lookup` a `kedvenc_kiado` mezőt a `konyv` gyűjtemény `kiado` mezőjével párosította, így egyetlen pipeline-ban valósítottam meg a JOIN-t és a szűrést.

Módosítás és törlés. A `k2`-es könyv címét `updateOne` + `$set` kombinációval módosítottam, a `k4`-es könyvet `deleteOne`-nal töröltem, végül a 30 év alatti tagokat `deleteMany`-vel eltávolítottam.

A teljes Java forráskód a mellékelt `Main_sajat.java` fájlban található.

A futtatás során a program a várt kimeneteket produkálta: a `find` az összes dokumentumot JSON formátumban írta ki, az `_id` alapú keresés pontosan egy találatot adott, a `lookup` pipeline sikeresen összekapcsolta a tagot a kiadó könyveivel, az átlagos életkor pedig 34.67-nek adódott.

Mellékletek

- `konyvek.json`, `tagok.json` – saját JSON adatfájlok (mongosh-hoz)
- `sajat.txt` – mongosh lekérdezések teljes forrása
- `Main.java` – Java MongoDB Driver implementáció
- `ettermek.json`, `szakacsok.json` – órai gyakorlat adatfájljai
- `Presentation.pdf` – prezentáció diái
- `Exercise.pdf` – feladatsor
- `Solution.pdf` – feladatsor megoldásai